Database System Engineering and Distributed Backend Development

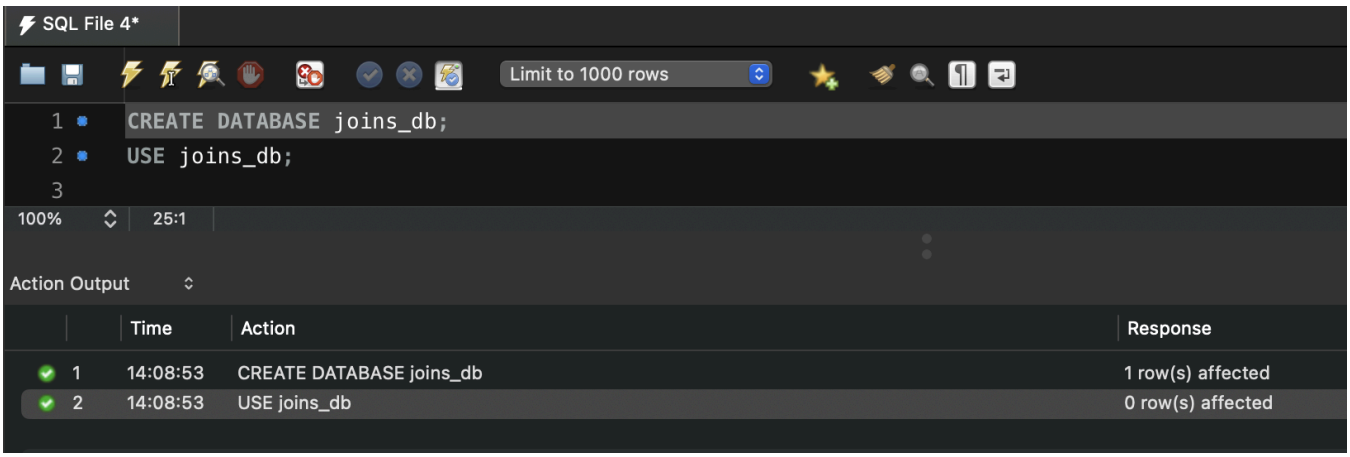
JOINS AND SET OPERATIONS

Name: Hansika
Roll No: 2520030237
Section: 7
Email: gunapatihansikareddy@klh.edu.in

Step 1 — Create and Select a Database

Query:

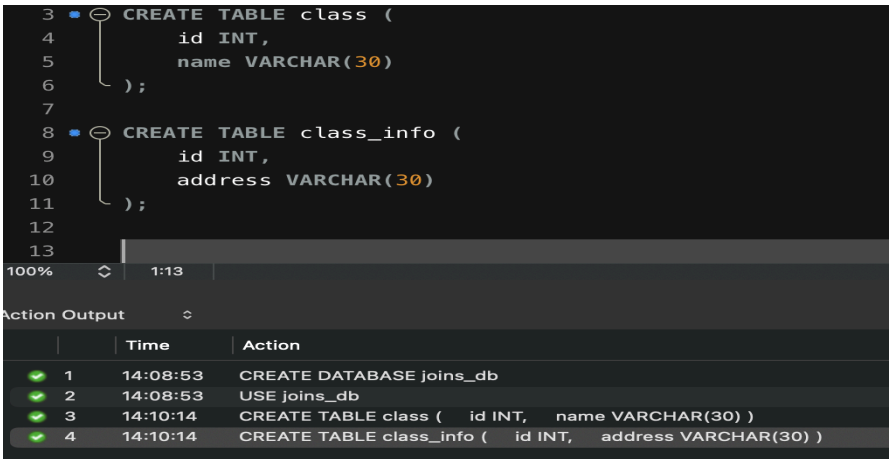
```
CREATE DATABASE joins_db;
USE joins_db;
```



Step 2 — Create the Tables (class, class_info)

Query:

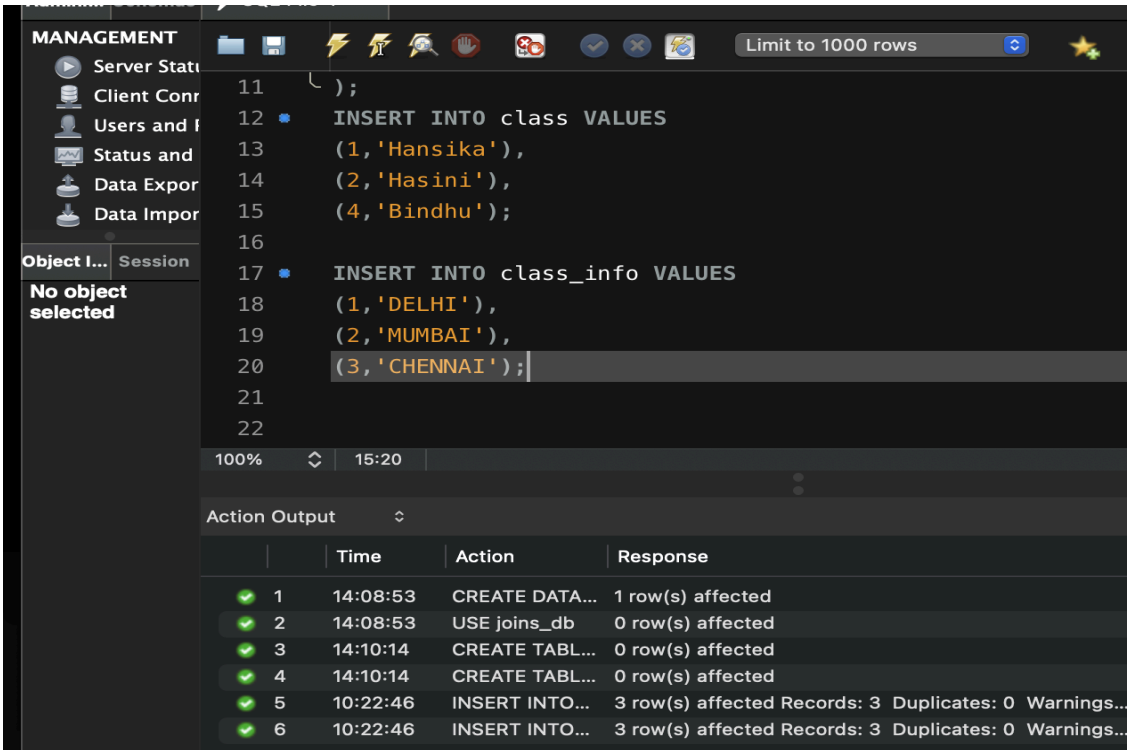
```
CREATE TABLE class (
    id INT,
    name VARCHAR(30)
);

CREATE TABLE class_info (
    id INT,
    address VARCHAR(30)
);
```



Step 3 — Insert Data

Query:

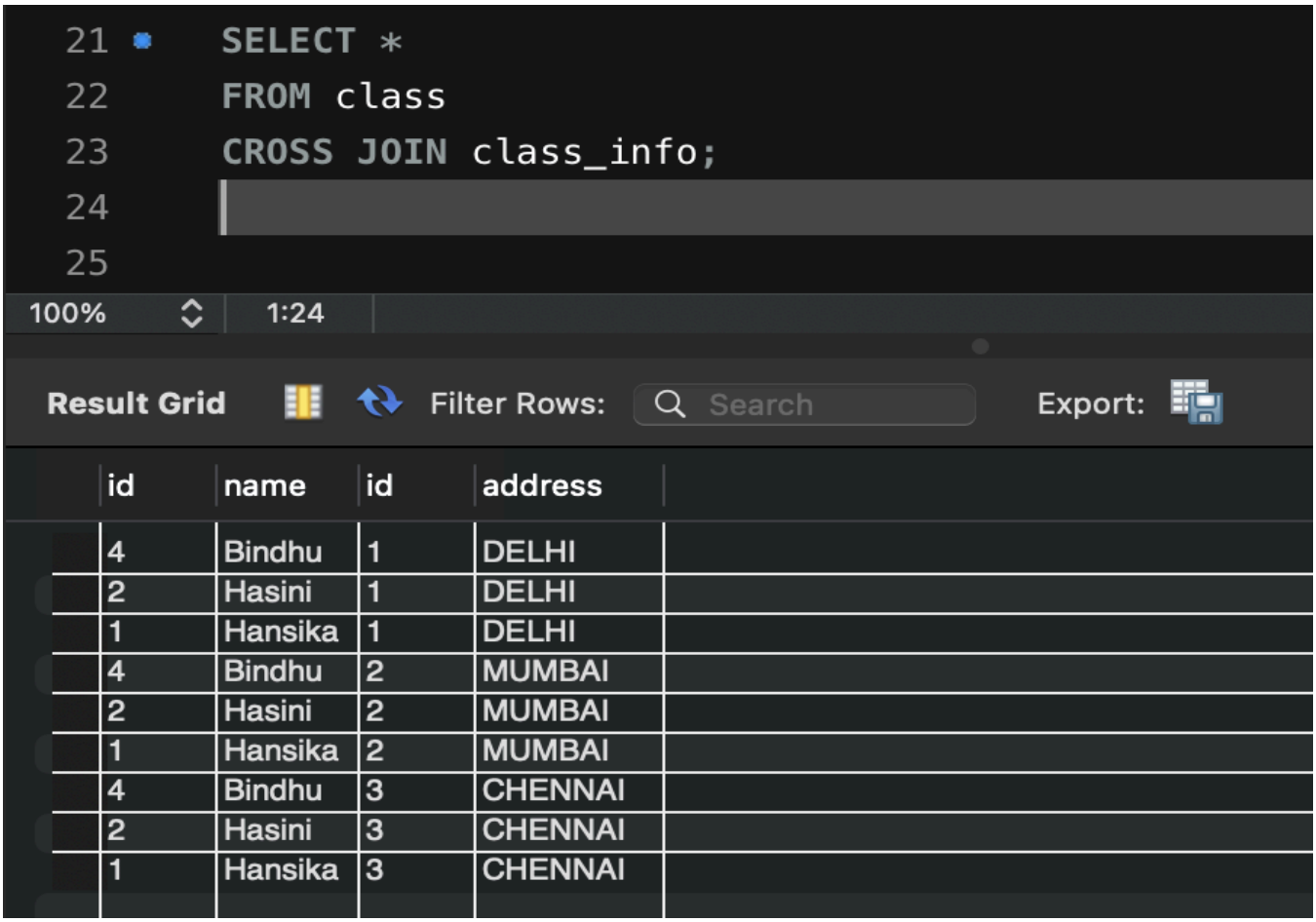
```
INSERT INTO class VALUES
(1,'Hansika'),
(2,'Hasini'),
(4,'Bindhu');

INSERT INTO class_info VALUES
(1,'DELHI'),
(2,'MUMBAI'),
(3,'CHENNAI');
```



Step 4 — Question 1: CROSS JOIN

Query:

```
SELECT *
FROM class
CROSS JOIN class_info;
```



```
21  SELECT *
22  FROM class
23  CROSS JOIN class_info;
24
25
```

100% 1:24

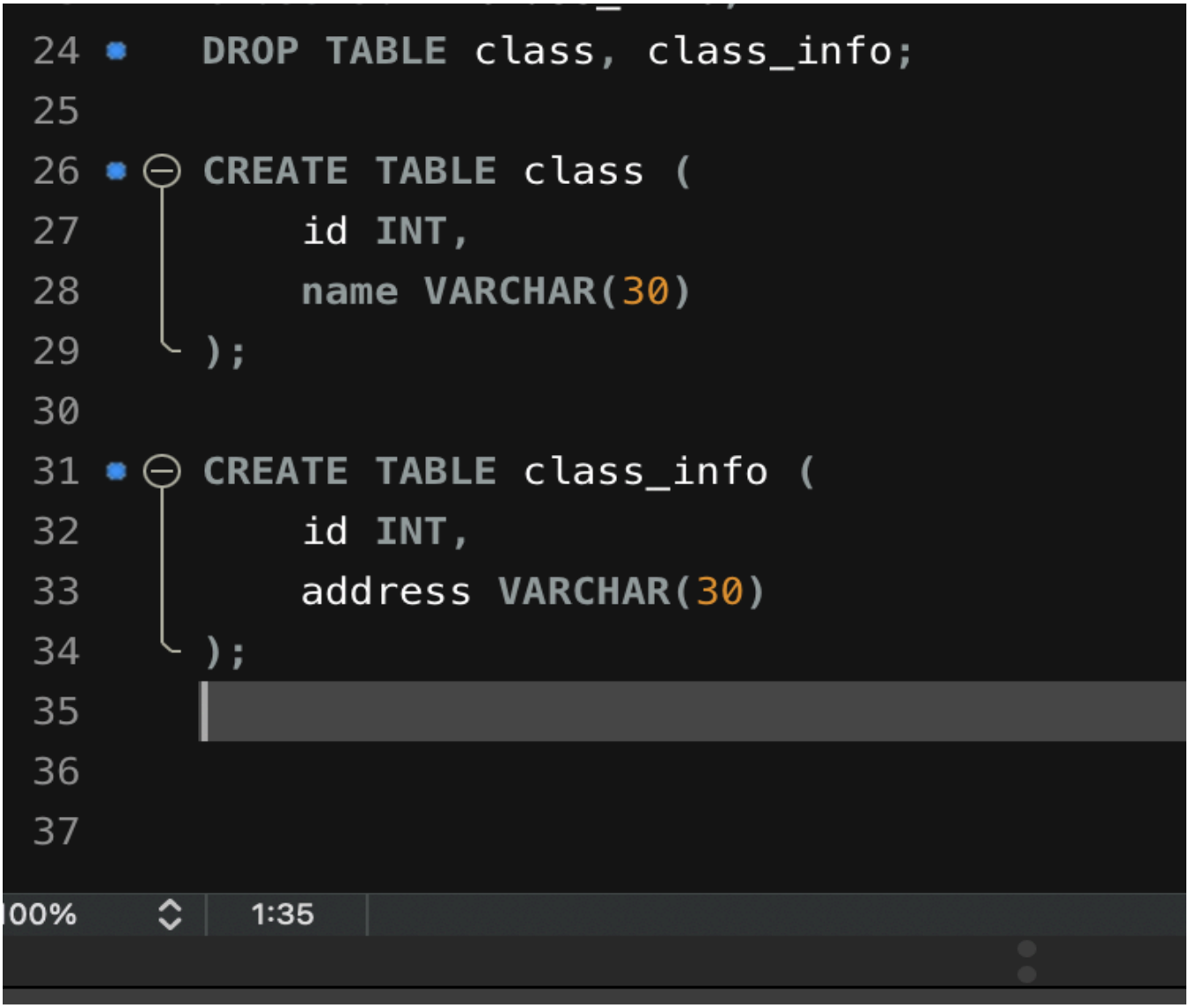
Result Grid | Filter Rows: Search | Export:

id	name	id	address
4	Bindhu	1	DELHI
2	Hasini	1	DELHI
1	Hansika	1	DELHI
4	Bindhu	2	MUMBAI
2	Hasini	2	MUMBAI
1	Hansika	2	MUMBAI
4	Bindhu	3	CHENNAI
2	Hasini	3	CHENNAI
1	Hansika	3	CHENNAI

Step 5 — Recreate Tables for INNER JOIN Section

Query:

```
DROP TABLE class, class_info;

CREATE TABLE class (
    id INT,
    name VARCHAR(30)
);

CREATE TABLE class_info (
    id INT,
    address VARCHAR(30)
);
```



Step 6 — Insert Updated Data

Query:

```
INSERT INTO class VALUES
(1,'Hansika'),
(2,'Hasini'),
(3,'Bindhu'),
(4,'Anusri');

INSERT INTO class_info VALUES
(1,'DELHI'),
(2,'MUMBAI'),
(3,'CHENNAI');
```

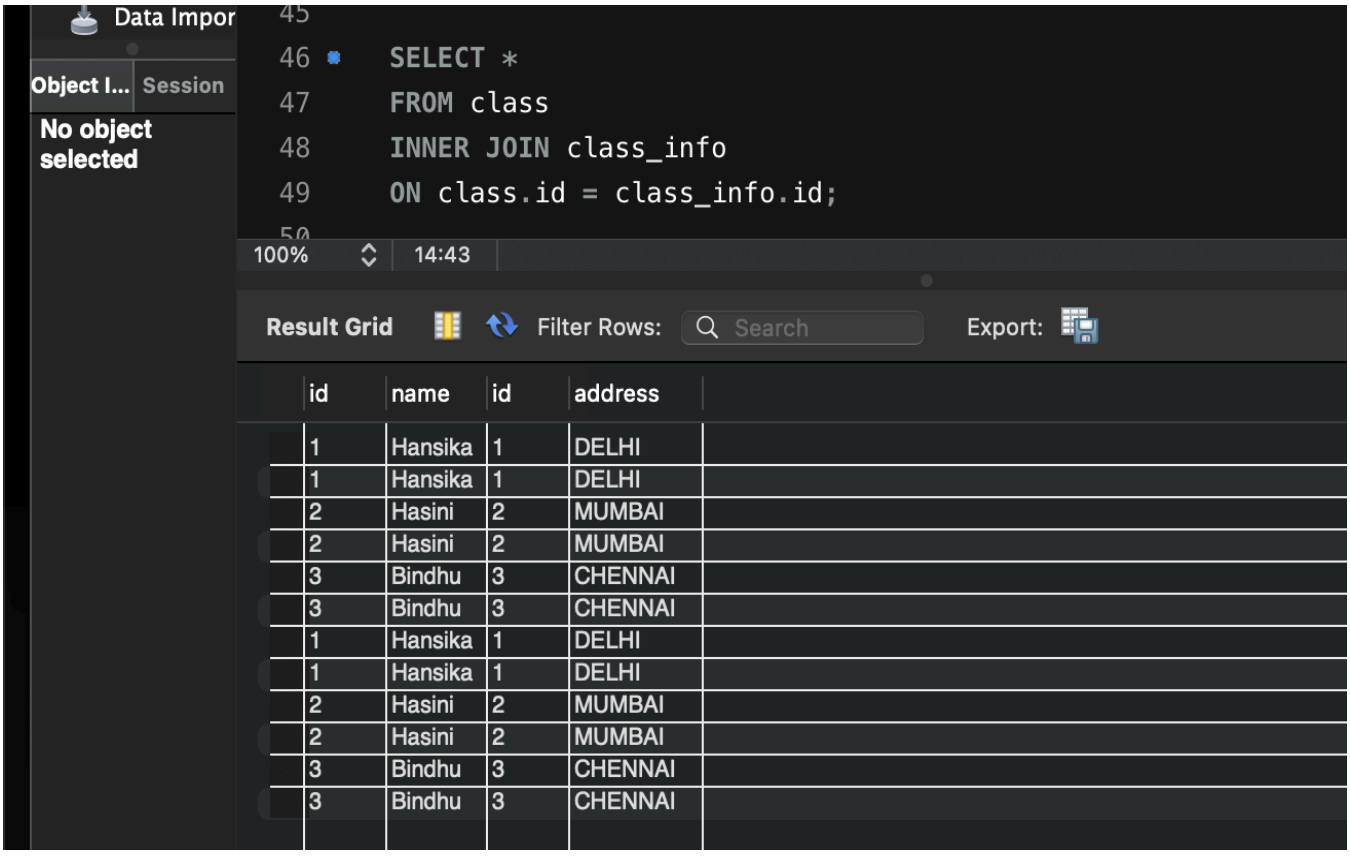
```
35    INSERT INTO class VALUES
36    (1,'Hansika'),
37    (2,'Hasini'),
38    (3,'Bindhu'),
39    (4,'Anusri');
40
41    INSERT INTO class_info VALUES
42    (1,'DELHI'),
43    (2,'MUMBAI'),
44    (3,'CHENNAI');
```

100% 12:32

Step 7 — Question 2: INNER JOIN

Query:

```
SELECT *
FROM class
INNER JOIN class_info
ON class.id = class_info.id;
```

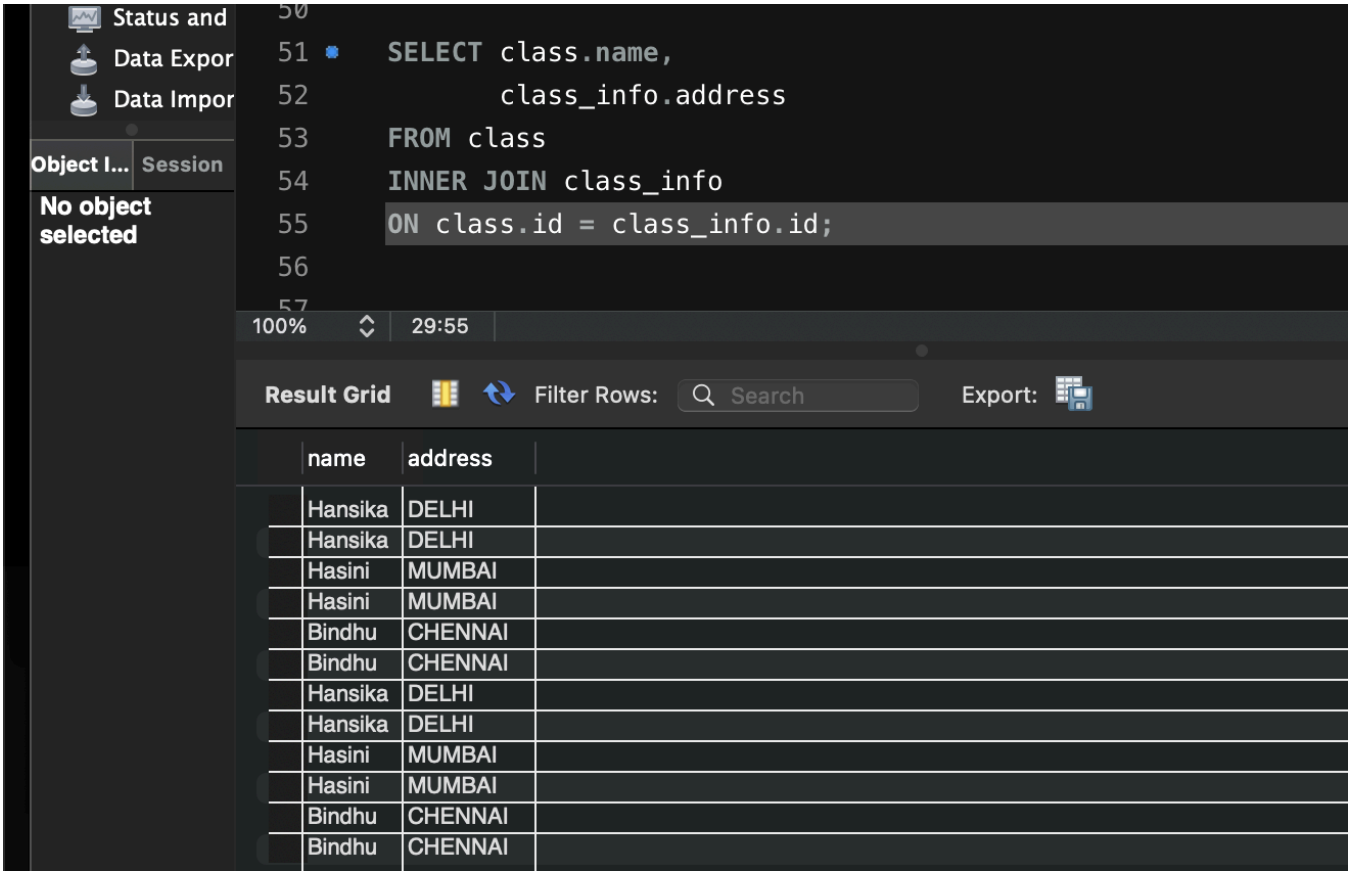


id	name	id	address
1	Hansika	1	DELHI
1	Hansika	1	DELHI
2	Hasini	2	MUMBAI
2	Hasini	2	MUMBAI
3	Bindhu	3	CHENNAI
3	Bindhu	3	CHENNAI
1	Hansika	1	DELHI
1	Hansika	1	DELHI
2	Hasini	2	MUMBAI
2	Hasini	2	MUMBAI
3	Bindhu	3	CHENNAI
3	Bindhu	3	CHENNAI

Step 8 — Question 3: INNER JOIN (Selected Columns)

Query:

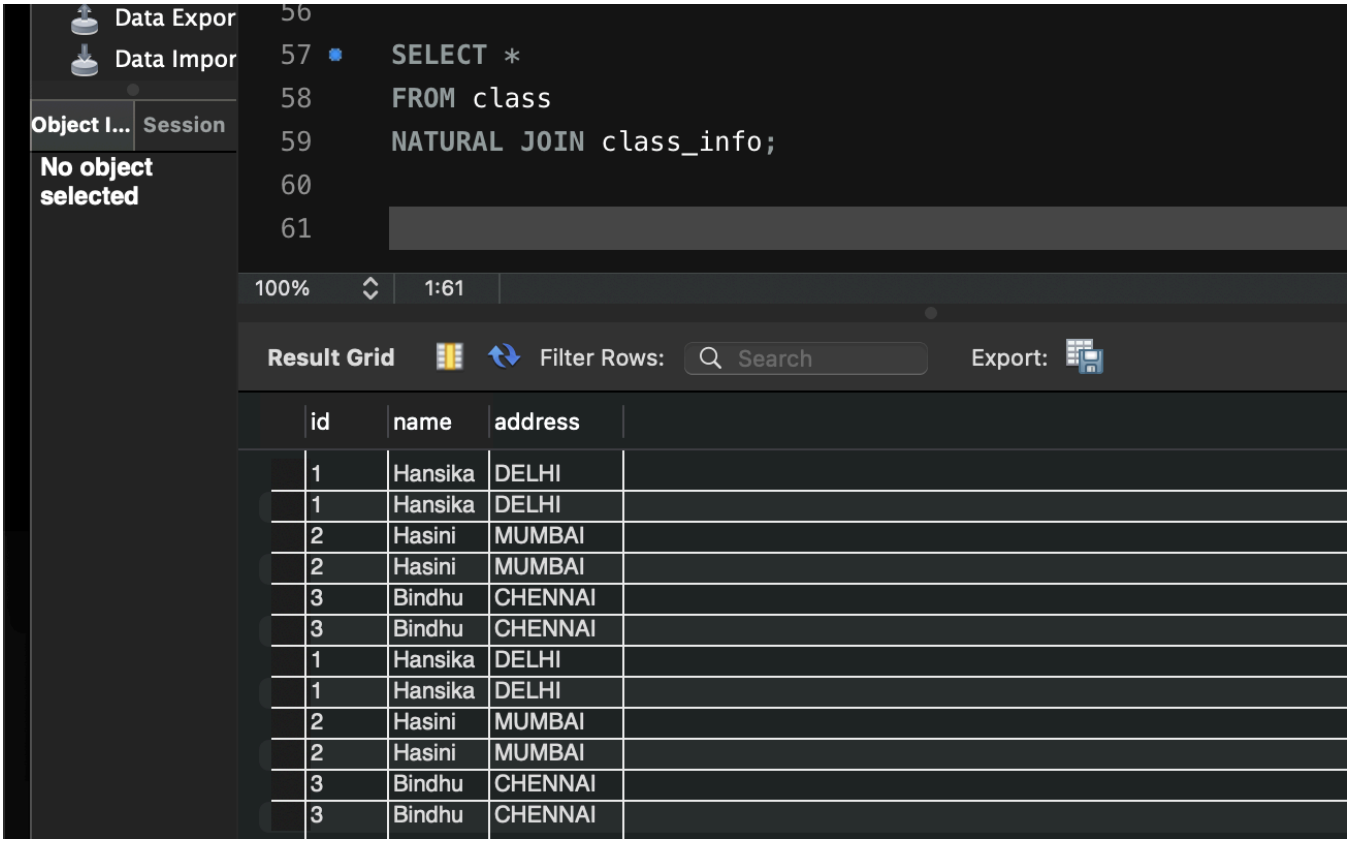
```
SELECT class.name,
       class_info.address
FROM class
INNER JOIN class_info
ON class.id = class_info.id;
```



Step 9 — Question 4: NATURAL JOIN

Query:

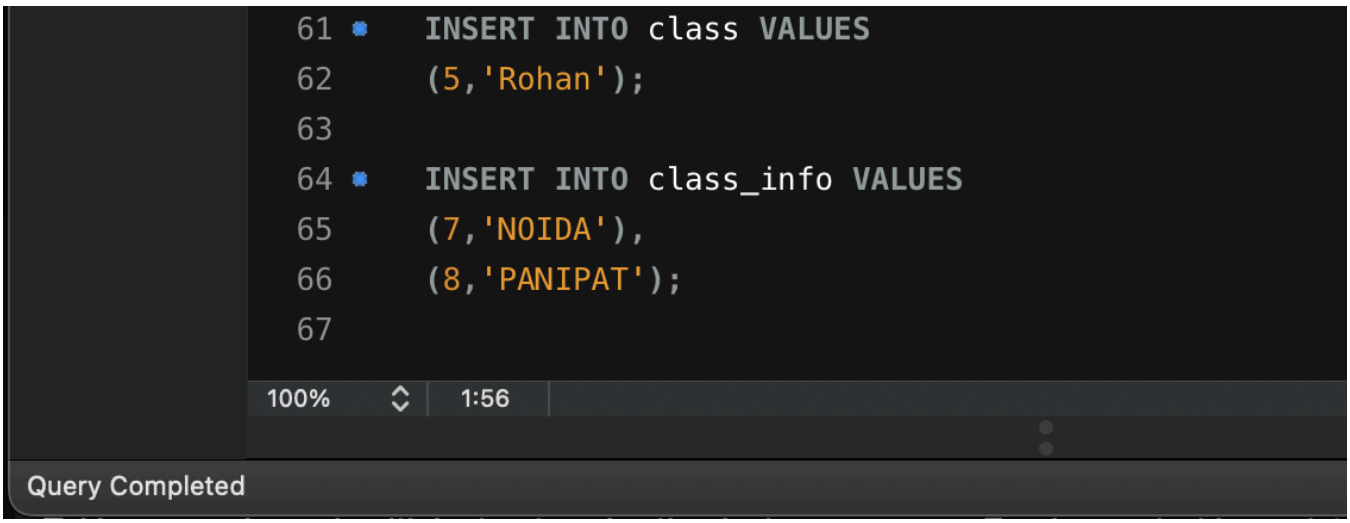
```
SELECT *
FROM class
NATURAL JOIN class_info;
```



Step 10 — Insert Extra Rows for Outer Join Section

Query:

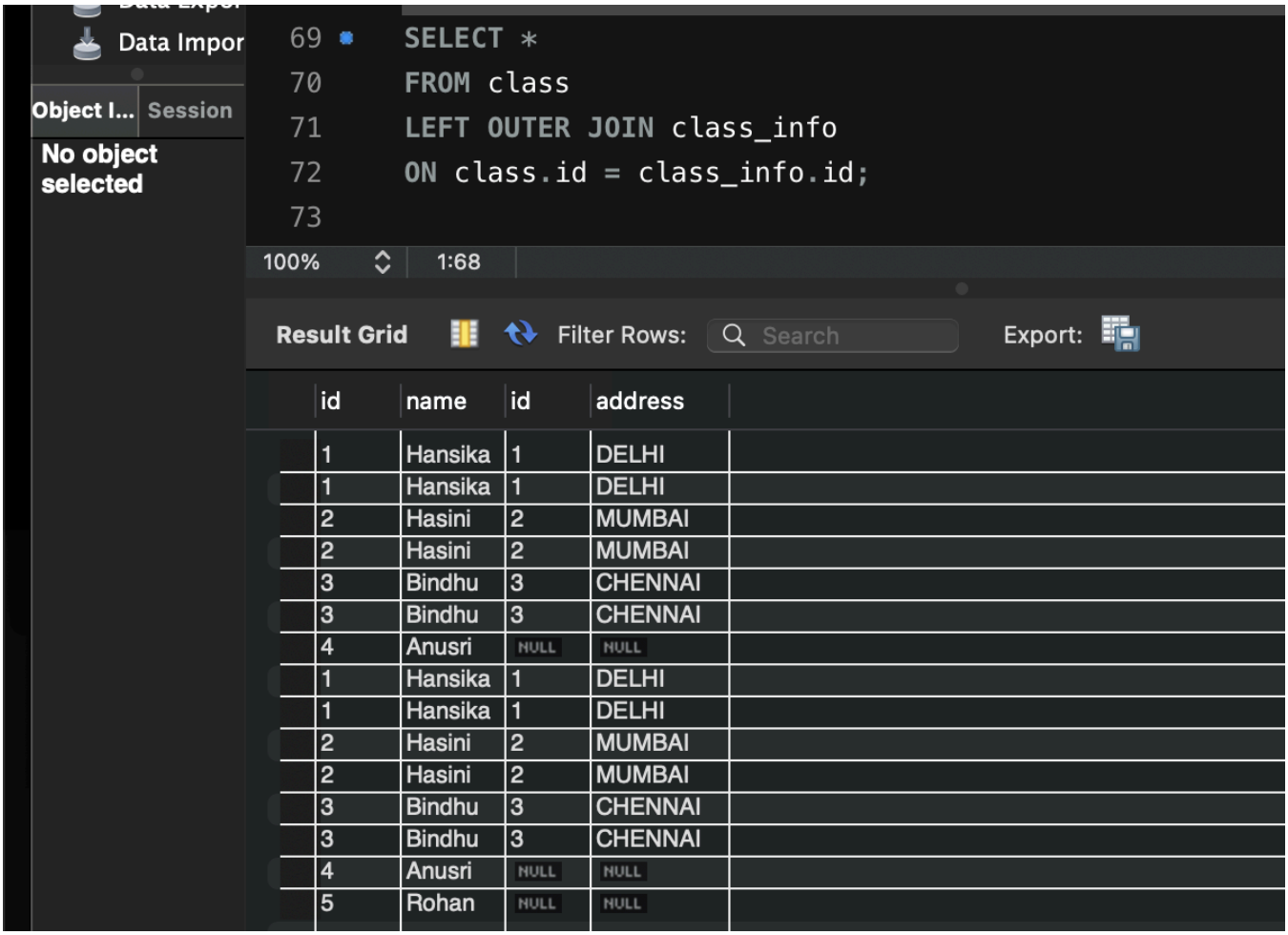
```
INSERT INTO class VALUES
(5,'Rohan');

INSERT INTO class_info VALUES
(7,'NOIDA'),
(8,'PANIPAT');
```



Step 11 — Question 5: LEFT OUTER JOIN

Query:

```
SELECT *
FROM class
LEFT OUTER JOIN class_info
ON class.id = class_info.id;
```

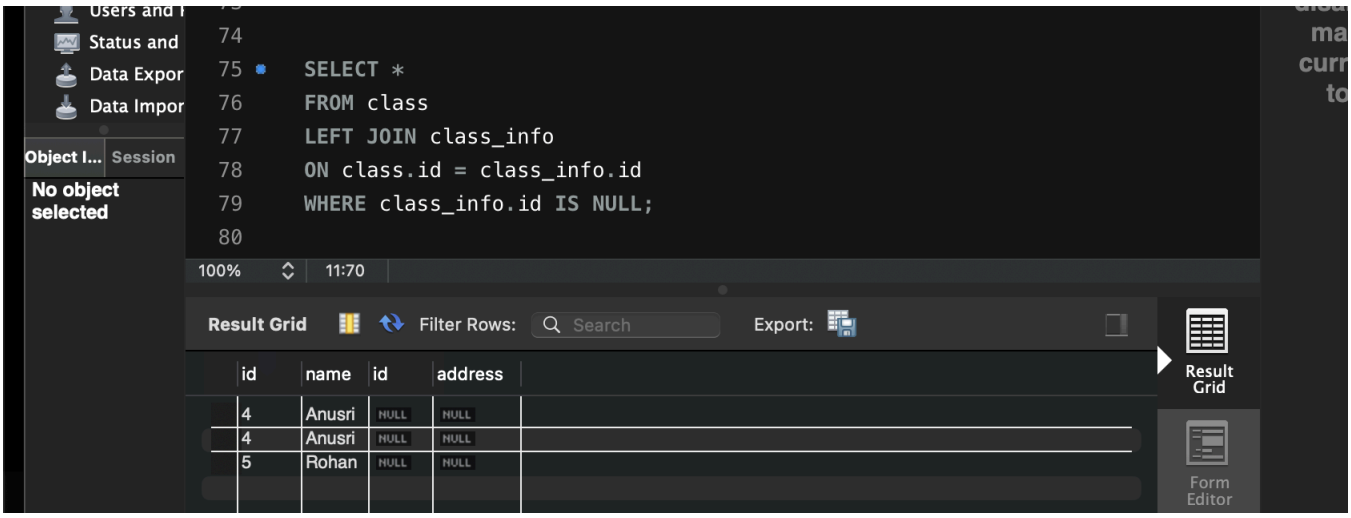


id	name	id	address
1	Hansika	1	DELHI
1	Hansika	1	DELHI
2	Hasini	2	MUMBAI
2	Hasini	2	MUMBAI
3	Bindhu	3	CHENNAI
3	Bindhu	3	CHENNAI
4	Anusri	NULL	NULL
1	Hansika	1	DELHI
1	Hansika	1	DELHI
2	Hasini	2	MUMBAI
2	Hasini	2	MUMBAI
3	Bindhu	3	CHENNAI
3	Bindhu	3	CHENNAI
4	Anusri	NULL	NULL
5	Rohan	NULL	NULL

Step 12 — Question 6: LEFT JOIN (Unmatched Only)

Query:

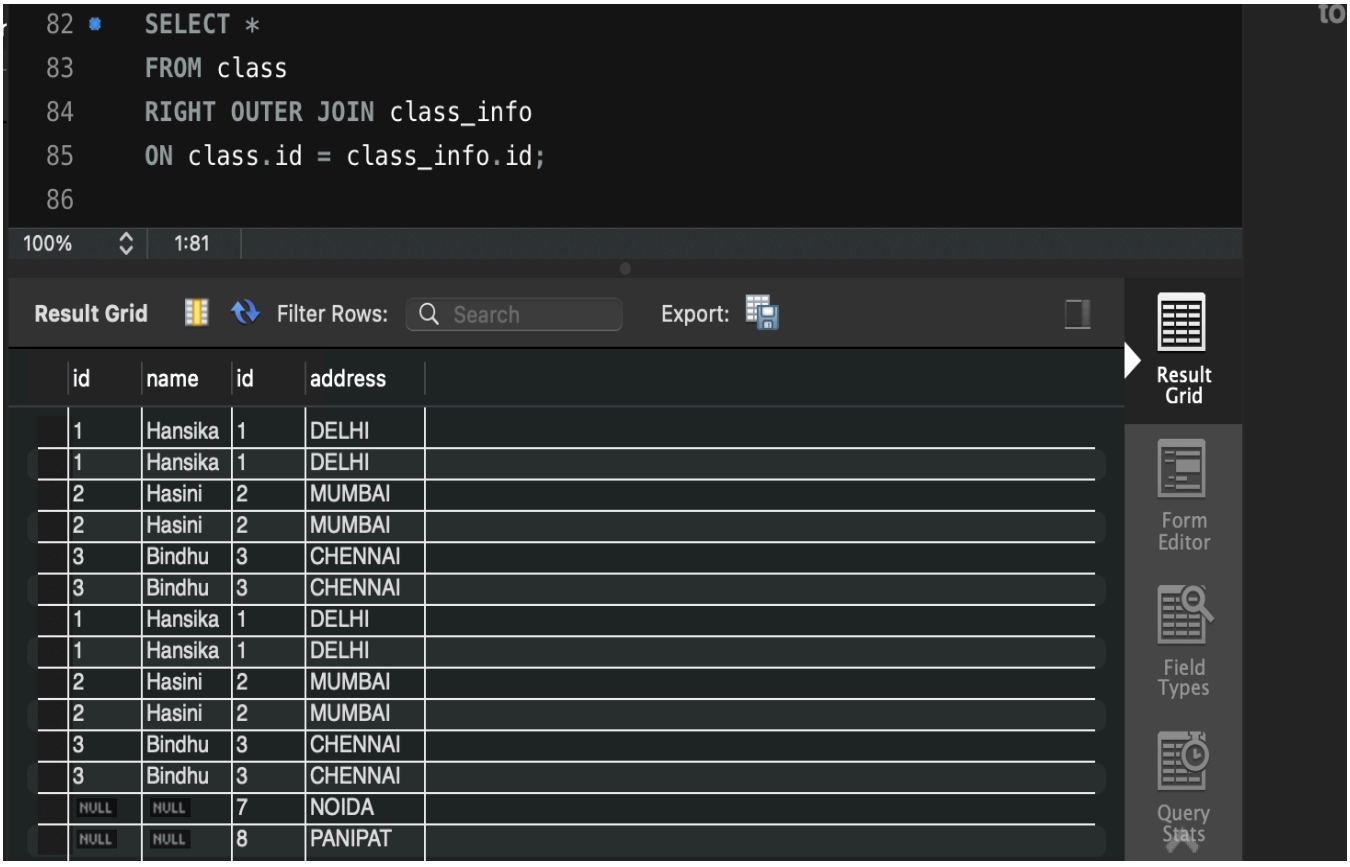
```
SELECT *
FROM class
LEFT JOIN class_info
ON class.id = class_info.id
WHERE class_info.id IS NULL;
```



Step 13 — Question 7: RIGHT OUTER JOIN

Query:

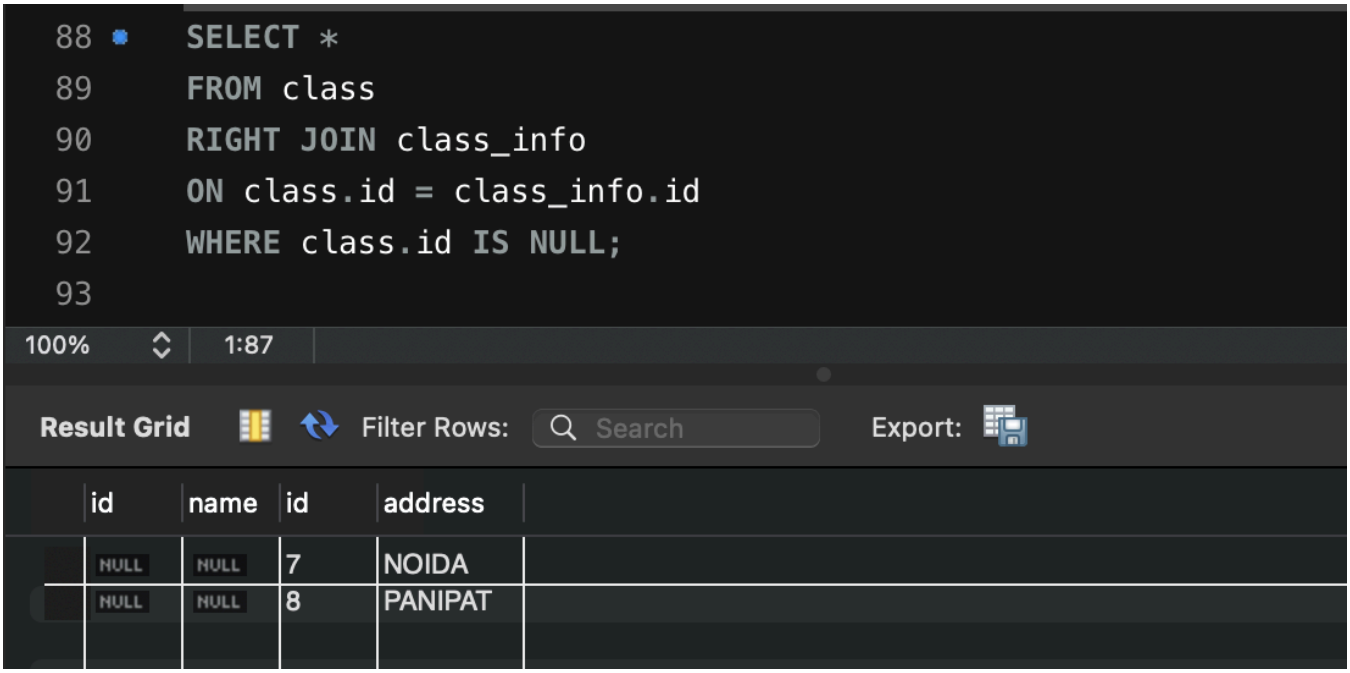
```
SELECT *
FROM class
RIGHT OUTER JOIN class_info
ON class.id = class_info.id;
```



Step 14 — Question 8: RIGHT JOIN (Unmatched Only)

Query:

```
SELECT *
FROM class
RIGHT JOIN class_info
ON class.id = class_info.id
WHERE class.id IS NULL;
```



Step 15 — Question 9: FULL OUTER JOIN

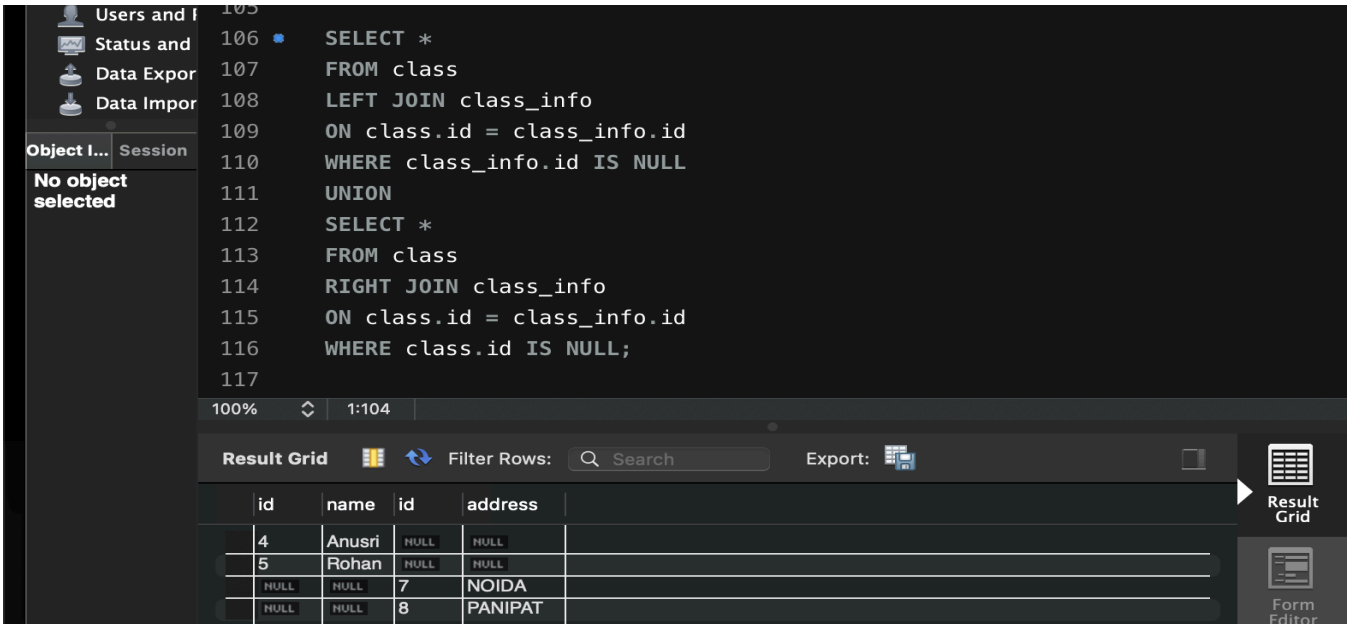
Query:

```
SELECT *
FROM class
LEFT JOIN class_info
ON class.id = class_info.id
UNION
SELECT *
FROM class
RIGHT JOIN class_info
ON class.id = class_info.id;
```

Step 16 — Question 10: FULL OUTER JOIN (Unmatched Only)

Query:

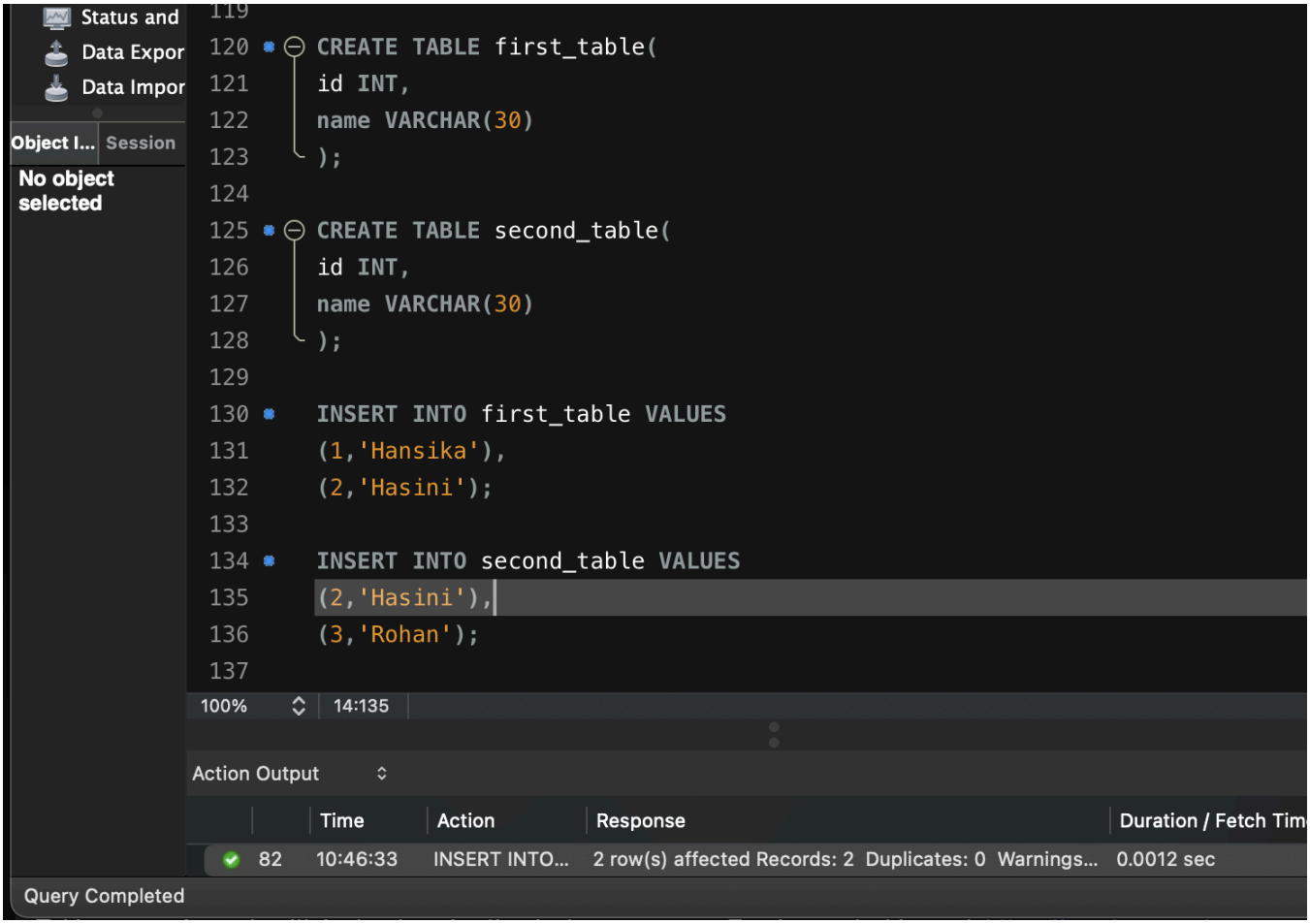
```
SELECT *
FROM class
LEFT JOIN class_info
ON class.id = class_info.id
WHERE class_info.id IS NULL
UNION
SELECT *
FROM class
RIGHT JOIN class_info
ON class.id = class_info.id
WHERE class.id IS NULL;
```



Step 17 — Create first_table and second_table (for UNION section)

Query:

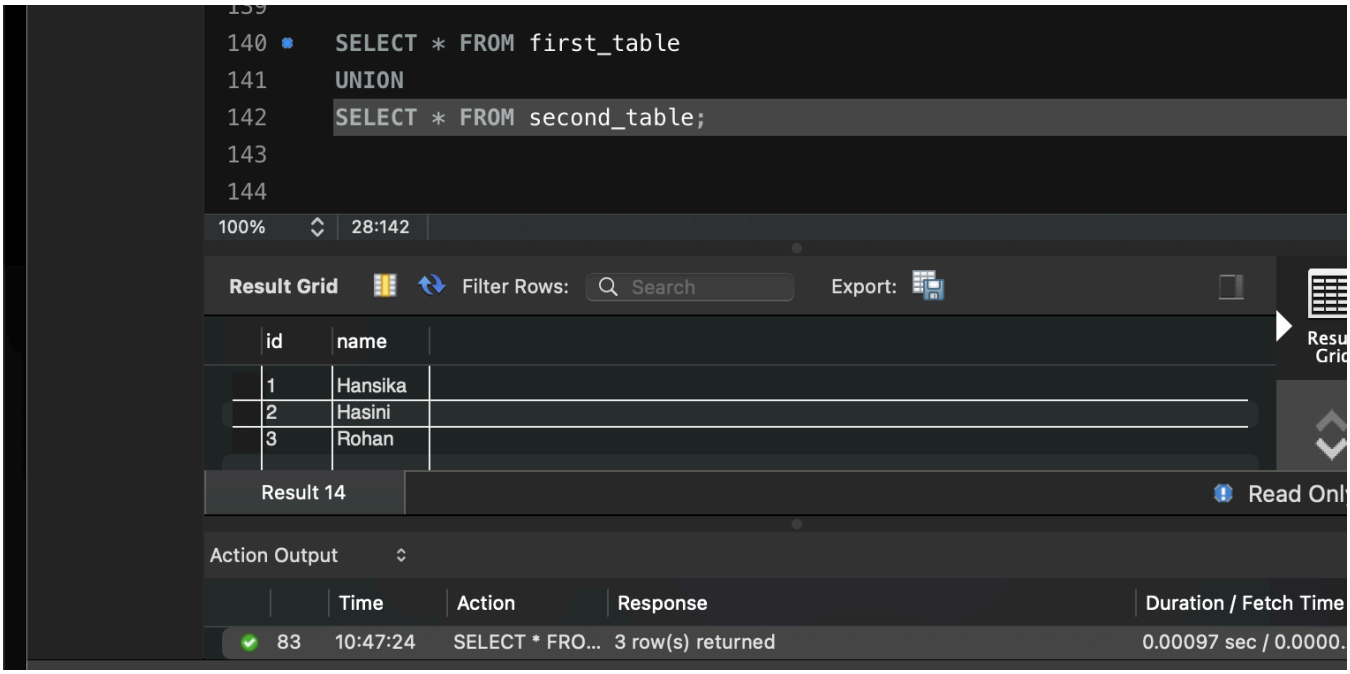
```
CREATE TABLE first_table(
id INT,
name VARCHAR(30)
);

CREATE TABLE second_table(
id INT,
name VARCHAR(30)
);

INSERT INTO first_table VALUES
(1,'Hansika'),
(2,'Hasini');

INSERT INTO second_table VALUES
(2,'Hasini'),
(3,'Rohan');
```



Step 18 — Question 11: UNION

Query:

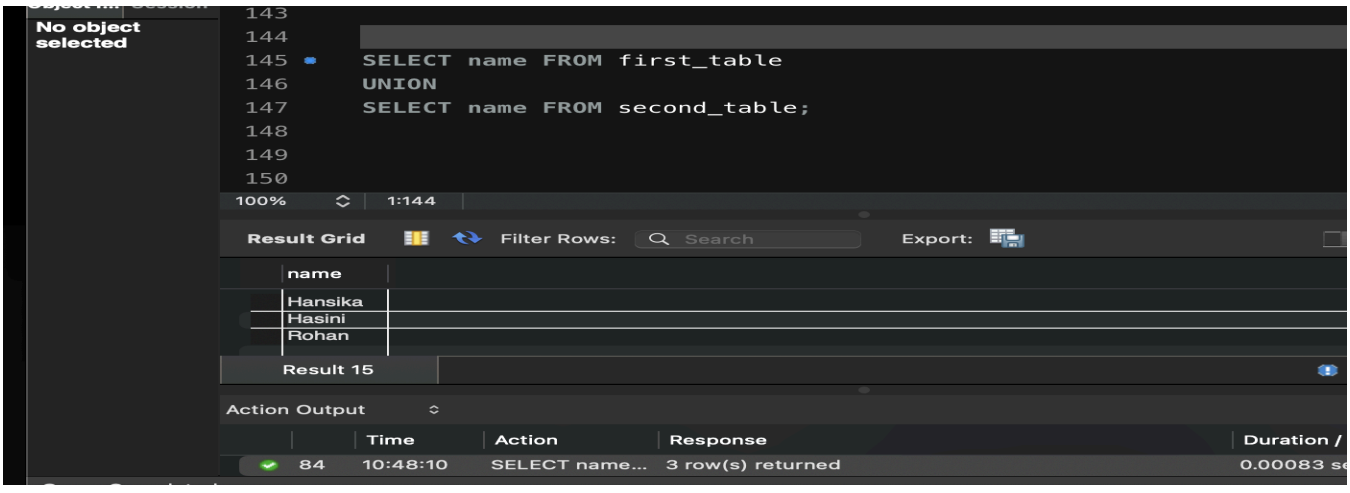
```
SELECT * FROM first_table
UNION
SELECT * FROM second_table;
```



Step 19 — Question 12: UNION (Names Only)

Query:

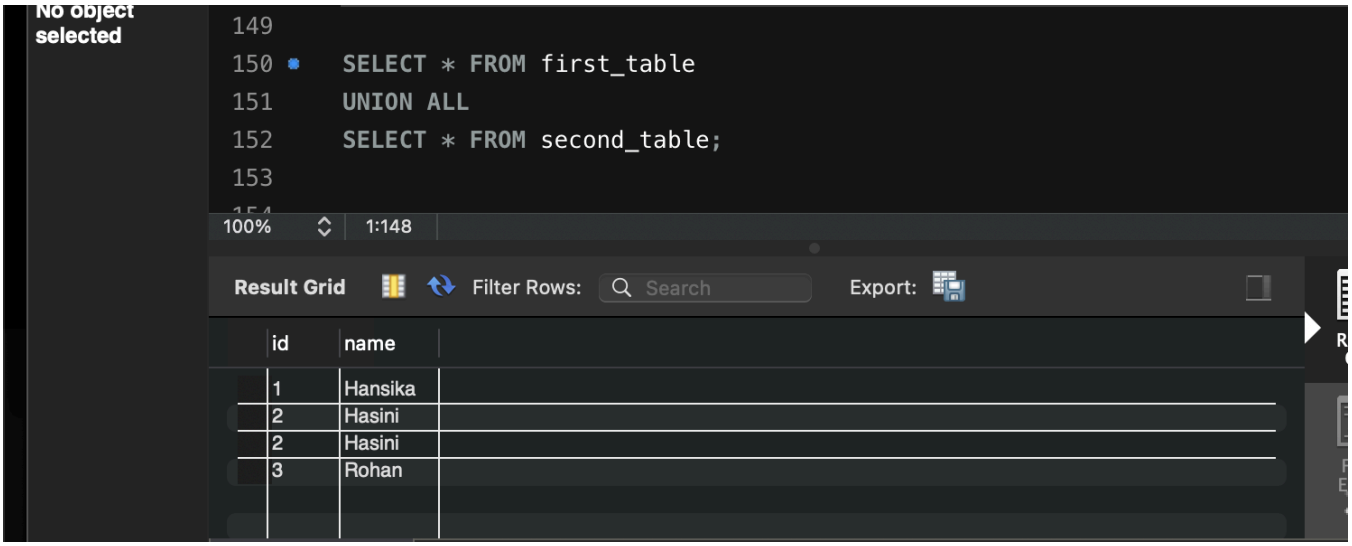
```
SELECT name FROM first_table
UNION
SELECT name FROM second_table;
```



Step 20 — Question 13: UNION ALL

Query:

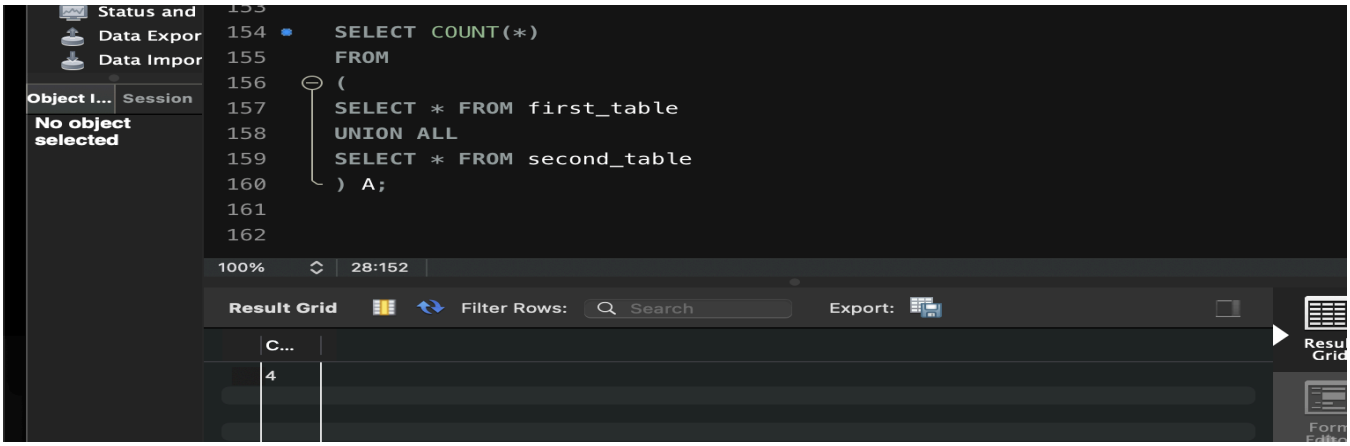
```
SELECT * FROM first_table
UNION ALL
SELECT * FROM second_table;
```



Step 21 — Question 14: UNION ALL (Row Count)

Query:

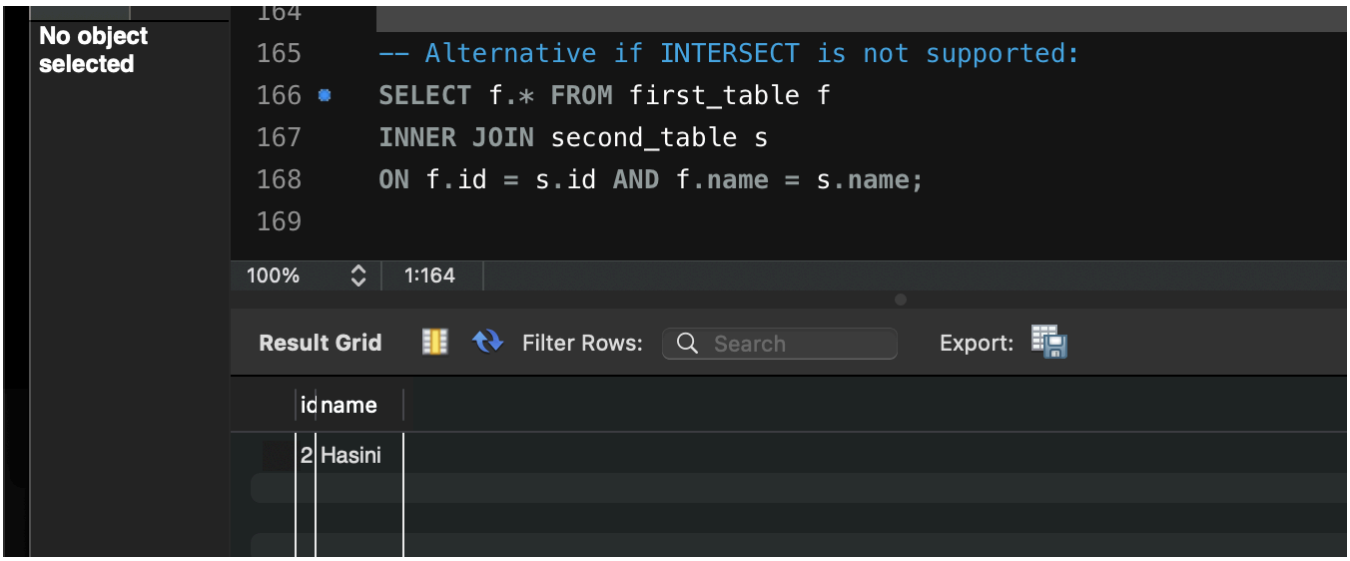
```
SELECT COUNT(*)
FROM
(
SELECT * FROM first_table
UNION ALL
SELECT * FROM second_table
) A;
```



Step 22 — Question 15: INTERSECT

Query:

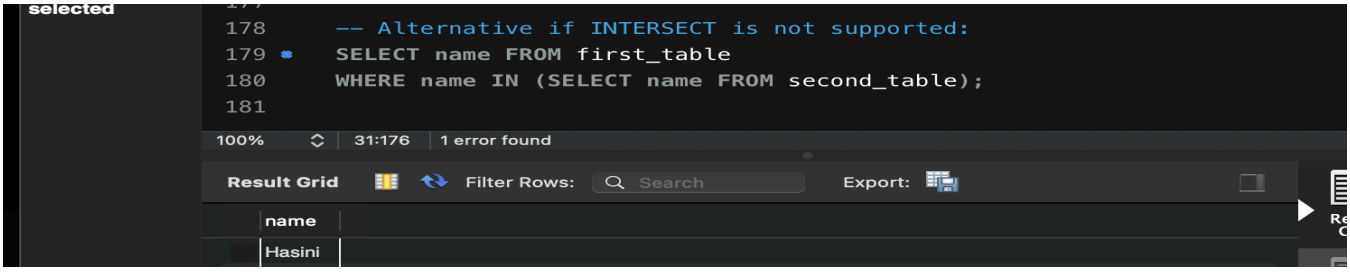
```
SELECT * FROM first_table
INTERSECT
SELECT * FROM second_table;

-- Alternative if INTERSECT is not supported:
SELECT f.* FROM first_table f
INNER JOIN second_table s
ON f.id = s.id AND f.name = s.name;
```



Step 23 — Question 16: INTERSECT (Names Only)

Query:

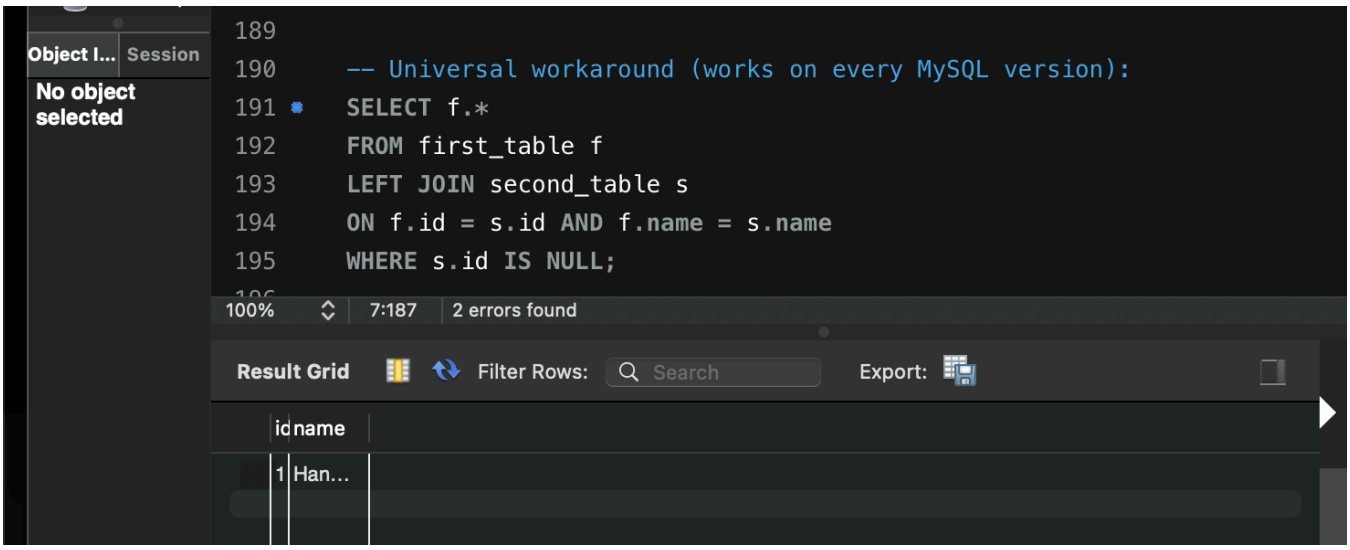
```
SELECT name FROM first_table
INTERSECT
SELECT name FROM second_table;

-- Alternative if INTERSECT is not supported:
SELECT name FROM first_table
WHERE name IN (SELECT name FROM second_table);
```



Step 24 — Question 17: MINUS

Query:

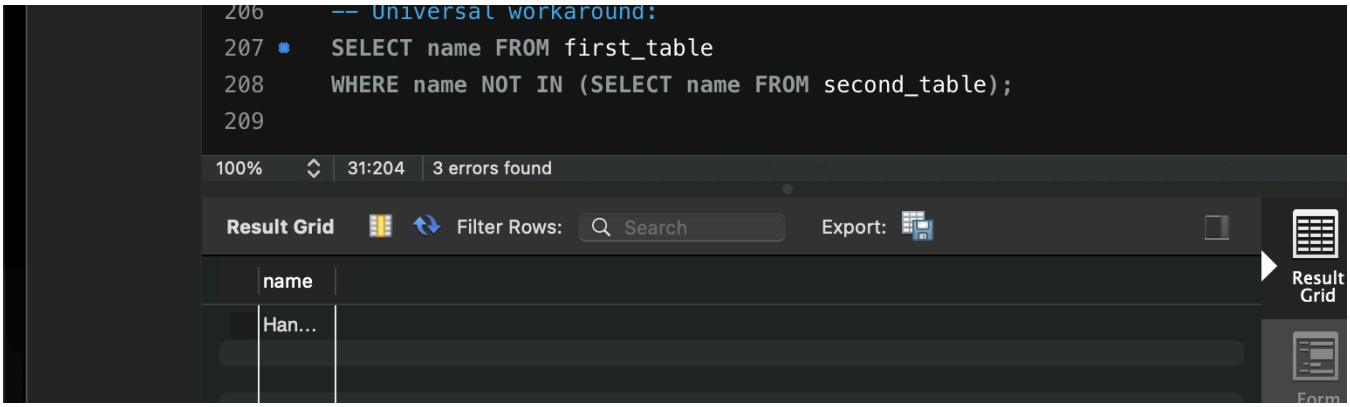
```
-- If your MySQL version supports EXCEPT:
SELECT * FROM first_table
EXCEPT
SELECT * FROM second_table;

-- Universal workaround (works on every MySQL version):
SELECT f.*
FROM first_table f
LEFT JOIN second_table s
ON f.id = s.id AND f.name = s.name
WHERE s.id IS NULL;
```



Step 25 — Question 18: MINUS (Names Only)

Query:

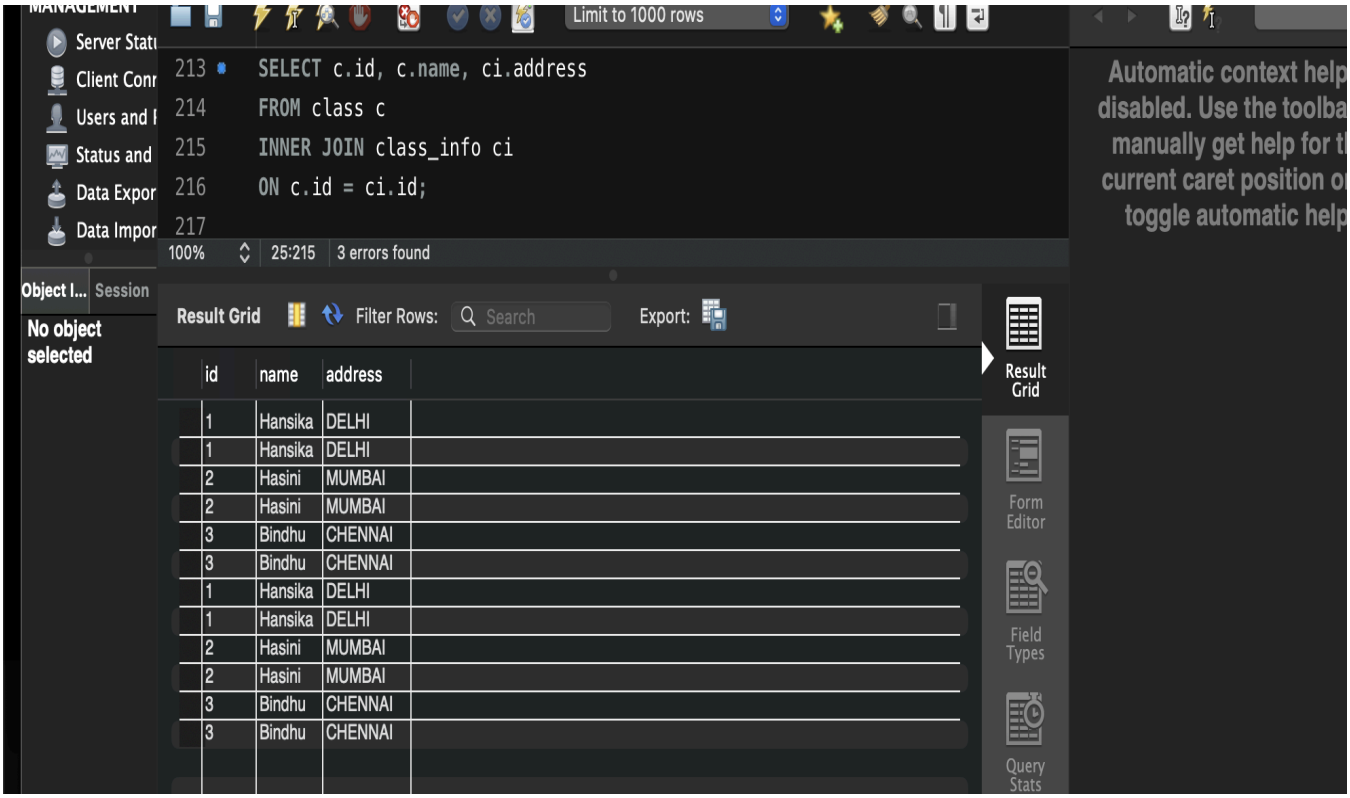
```
-- If your MySQL version supports EXCEPT:
SELECT name FROM first_table
EXCEPT
SELECT name FROM second_table;

-- Universal workaround:
SELECT name FROM first_table
WHERE name NOT IN (SELECT name FROM second_table);
```



Step 26 — Question 19: Advanced — Matching Addresses

Query:

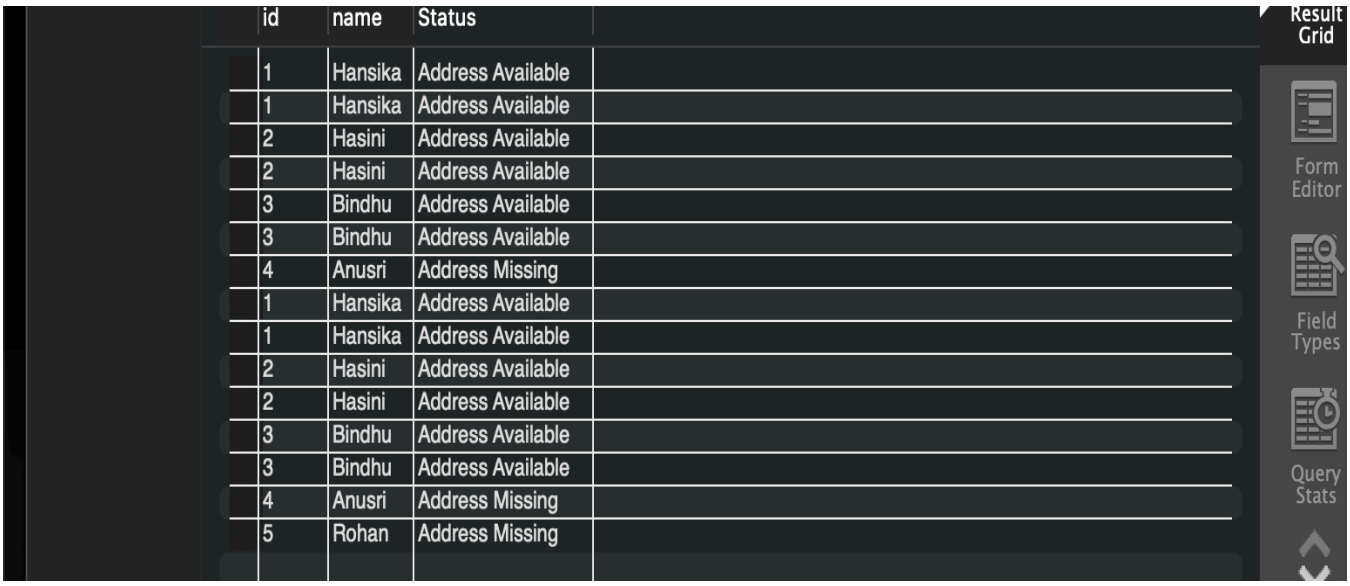
```
SELECT c.id, c.name, ci.address
FROM class c
INNER JOIN class_info ci
ON c.id = ci.id;
```



Step 27 — Question 20: Advanced — Address Availability Report

Query:

```
SELECT c.id,
       c.name,
       CASE
           WHEN ci.address IS NULL
           THEN 'Address Missing'
           ELSE 'Address Available'
       END AS Status
FROM class c
LEFT JOIN class_info ci
ON c.id = ci.id;
```



id	name	Status
1	Hansika	Address Available
1	Hansika	Address Available
2	Hasini	Address Available
2	Hasini	Address Available
3	Bindhu	Address Available
3	Bindhu	Address Available
4	Anusri	Address Missing
1	Hansika	Address Available
1	Hansika	Address Available
2	Hasini	Address Available
2	Hasini	Address Available
3	Bindhu	Address Available
3	Bindhu	Address Available
4	Anusri	Address Missing
5	Rohan	Address Missing